PROBLEM STATEMENT

Sana ur Rehman Arain – sana.arain.tech@gmail.com

Contents

Project Track 1
Problem statement 1
Goals and Objectives 1
Credit Risk Pipeline Flowchart 3
Code Design Strategy 4
Algorithm 5

Project Track

This work adheres to the Practical Research Track in the field of Financial Machine Learning and Risk Management. This project aims at designing, validating, and describing an end-to-end pipeline for credit risk classification with the help of the OpenIntro Loans dataset.

Problem statement

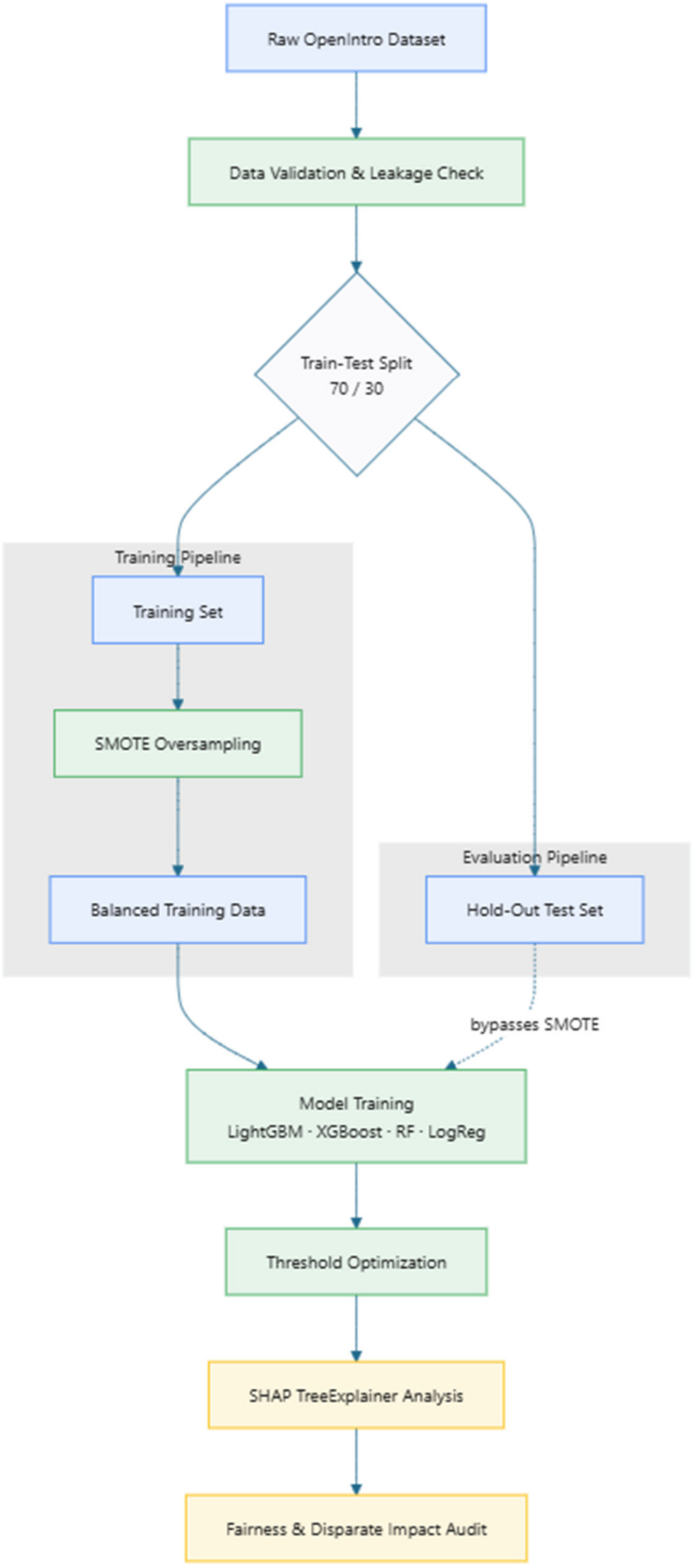
- **The Business Need:** The evaluation of credit risk in particular the probability of default (PD) is a crucial component of modern banking and financial risk management.
- **The Limitations of the Traditional Methods:** In accordance with the Basel III framework, traditional statistical approaches (such as Logistic Regression) had been used in banking due to their transparency and "monotonicity" properties. However, traditional models are inefficient in case of high dimensionality and inability to find complex non-linear relations in borrower profile.
- **The Machine Learning Challenge:** The development of machine learning (ML) methods (e.g., gradient boosting) provides much better prediction power. However, such models are viewed as "Black Boxes" by financial regulators (such as the Federal Reserve SR 11-7).
- **The Fundamental Challenge:** Borrowers face a dilemma of choosing either a highly accurate ML model that cannot be explained (thereby violating fair lending and adverse action laws) or a traditional explainable but less effective one.

Goals and Objectives

- **Main Objective:** Designing a highly predictive credit risk model based on state-of-the-art gradient-boosting methods that will satisfy regulatory transparency (SR 11-7) and fair lending requirements.
- **Research Objective 1 (Comparing Model Predictions):** Investigating the prediction capacity of standard statistical benchmarks (Logistic Regression) and advanced ensemble algorithms (Random Forest, XGBoost, and LightGBM) for high dimensional borrowing data.
- **Research Objective 2 (Mitigating Imbalance Data Set):** Using the Synthetic Minority Over-sampling Technique (SMOTE) on the training set only to address the serious 1.8% imbalance without any artificial leakage of data.

- **Research Objective 3 (Applying Explainable AI):** Using the Shapley Additive Explanations (SHAP) through Tree Explainer to build the global explanations and case-level local attributions for adverse action report.
- **Research Objective 4 (Decision Threshold Calibration):** Fine-tuning the probability threshold in order to optimize precision-recall trade-off for detecting the minority class defaults.
- **Research Objective 5 (Ensuring Economic Fairness):** Analyzing economic fairness and disparate impact across income groups using the False Positive Rate parity to ensure compliance with fair lending principles.

Credit Risk Pipeline Flowchart



Code Design Strategy

```
# 1. Data Ingestion & Leakage Prevention

Load OpenIntro_Loans_Data

Drop post-origination columns (paid_principal, paid_total, etc.) to prevent
target leakage


# 2. Preprocessing & Splitting

Encode categorical variables using One-Hot Encoding

Split data into X_train, X_test, y_train, y_test (70/30 stratified split)


# 3. Class Imbalance Mitigation (Training Data Only)

Initialize SMOTE()

X_train_balanced, y_train_balanced = SMOTE.fit_resample(X_train, y_train)


# 4. Model Training

Initialize LightGBM_Classifier(random_state=42, objective='binary')

LightGBM_Classifier.fit(X_train_balanced, y_train_balanced)


# 5. Evaluation & Optimization

Generate predictions on X_test

Calculate ROC-AUC, Precision, and Recall

Determine optimal decision threshold maximizing F1/Cost-Sensitivity


# 6. Explainability & Fairness

Initialize SHAP_TreeExplainer(LightGBM_Classifier)

Calculate SHAP values for X_test to identify risk drivers (interest_rate,
DTI)

Calculate False Positive Rate across low-income vs. high-income proxy groups
```

Algorithm

```
ALGORITHM 1 — Credit Default Prediction with Fairness Audit

INPUT   D : OpenIntro Loans dataset
        A : protected attribute (income-bracket proxy)
OUTPUT  f*: fitted classifier      τ*: decision threshold
        Φ : SHAP attributions      R : fairness report

┌─ PHASE 1 — Ingestion & Leakage Control ──────────────────────────┐
  1.1   D  ← load(OpenIntro_Loans_Data)
  1.2   y  ← 1 if loan_status ∈ {Charged Off, Default} else 0
  1.3   L  ← {paid_principal, paid_interest, paid_total,
              paid_late_fees, loan_status}          ▷ post-origination
  1.4   X  ← D \ (L ∪ {y})
  1.5   ASSERT ∀ x ∈ X : observable_at(x) ≤ t₀      ▷ origination time
└──────────────────────────────────────────────────────────────────┘

┌─ PHASE 2 — Partitioning  (before any parameter is fitted) ───────┐
  2.1   X_tr, X_te, y_tr, y_te ← stratified_split(X, y, .30, seed=42)
  2.2   X_tr, X_va, y_tr, y_va ← stratified_split(X_tr, y_tr, .20, seed=42)
        ▷ X_va carries threshold selection; X_te is opened exactly once
└──────────────────────────────────────────────────────────────────┘

┌─ PHASE 3 — Preprocessing Pipeline  (fitted on X_tr only) ────────┐
  3.1   T_num ← MedianImputer ∘ StandardScaler        ▷ scaler for LogReg
  3.2   T_cat ← ModeImputer   ∘ OneHotEncoder(handle_unknown='ignore')
  3.3   T     ← ColumnTransformer(T_num, T_cat)
  3.4   T.fit(X_tr)                                  ▷ never on X_va/X_te
└──────────────────────────────────────────────────────────────────┘

┌─ PHASE 4 — Imbalance Mitigation  (inside CV folds only) ─────────┐
  4.1   S ← SMOTENC(categorical_features = idx_cat, seed = 42)
  4.2   P ← ImbPipeline([ T , S , classifier ])
        ▷ pipeline form guarantees resampling never leaks across folds
└──────────────────────────────────────────────────────────────────┘

┌─ PHASE 5 — Model Training & Selection ───────────────────────────┐
  5.1   M ← {LightGBM, XGBoost, RandomForest, LogisticRegression}
  5.2   FOR EACH m ∈ M:
  5.3       θ_m ← RandomisedSearchCV(P(m), StratifiedKFold(k=5),
                                     scoring = 'average_precision')
  5.4   f*  ← argmax_m  CV_score(θ_m)
  5.5   BASELINE: refit f* with class_weight='balanced' instead of SMOTE
        ▷ reports whether resampling actually earned its place
└──────────────────────────────────────────────────────────────────┘

┌─ PHASE 6 — Threshold Optimisation  (validation set) ─────────────┐
  6.1   p_va ← f*.predict_proba(X_va)[:, 1]
  6.2   τ*   ← argmax_τ  [ F1(y_va, p_va ≥ τ) ]
             or  argmin_τ [ C_FN·FN(τ) + C_FP·FP(τ) ]    ▷ cost-sensitive
└──────────────────────────────────────────────────────────────────┘
```

```
┌─ PHASE 7 — Final Evaluation  (held-out test set, single pass) ──────┐
  7.1   p_te ← f*.predict_proba(X_te)[:, 1]
  7.2   ŷ_te ← p_te ≥ τ*
  7.3   REPORT  PR-AUC, ROC-AUC, Precision, Recall, F1,
                Brier score, calibration curve, confusion matrix
        ▸ SMOTE distorts the prior — calibration must be checked
└─────────────────────────────────────────────────────────────────────┘

┌─ PHASE 8 — Explainability ──────────────────────────────────────────┐
  8.1   E ← SHAP.TreeExplainer(f*)
  8.2   Φ ← E.shap_values(X_te)
  8.3   RANK features by mean(|Φ|)      ▸ expect interest_rate, DTI, grade
  8.4   PLOT beeswarm + dependence(interest_rate, DTI)
        ▸ base value reflects the resampled prior, not population rate
└─────────────────────────────────────────────────────────────────────┘

┌─ PHASE 9 — Fairness Audit ──────────────────────────────────────────┐
  9.1   A ← income_bracket(X_te)        ▸ groups: low vs. high
  9.2   FOR EACH group a ∈ A:
  9.3       COMPUTE  FPR_a , FNR_a , TPR_a , selection_rate_a
  9.4   ΔEO  ← max_a TPR_a − min_a TPR_a            ▸ equal opportunity
  9.5   DI   ← min_a sel_a / max_a sel_a            ▸ 80% rule threshold
  9.6   R    ← {ΔEO, DI, per-group confusion matrices}
  9.7   IF DI < 0.80: FLAG disparate impact; document mitigation
└─────────────────────────────────────────────────────────────────────┘
```